Cheat Sheet

Classes, algorithms, data and git on two pages

Dyslexia-friendly edition (Lexend, wider spacing).

Classes and objects

```
class Pet:                    #
    ↪ blueprint
    def __init__(self, name):
        self.name = name  #
    ↪ attribute
        self.energy = 5

    def play(self):         # method
        self.energy -= 2
        return self.name + " had
    ↪ fun!"

mo = Pet("Mo")              # an
    ↪ object
print(mo.play(), mo.energy)
```

Attributes = what it has. **Methods** = what it does. `self` means “this object”; every method needs it first.

Inheritance (IS a), composition (HAS a)

```
class Dragon(Pet):          # IS a
    ↪ Pet
    def __init__(self, name):
        super().__init__(name)
        self.fire = 3

    def play(self):         #
    ↪ override
        return "Roar!"

class Zoo:                  # HAS
    ↪ pets
    def __init__(self):
        self.pets = [Pet("Mo"),
                     Dragon("Ash")]

    def play_all(self):
        for pet in self.pets:
            print(pet.play())
```

Searching: how fast is fast?

```
def binary_search(items, target):
    # items must be SORTED
    low, high = 0, len(items) - 1
    while low <= high:
        mid = (low + high) // 2
        if items[mid] == target:
            return mid    # found
    ↪ it
        elif items[mid] < target:
            low = mid + 1  # right
        else:
            high = mid - 1 # left
    return -1              # not
    ↪ there
```

Items	10	1000	1 million
Linear search, at most	10	1000	1 million
Binary search, at most	4	10	20

Sorting

Selection sort: find the smallest, swap it to the front, repeat with the rest. **Bubble sort:** swap neighbors in the wrong order; repeat until nothing swaps.

```
print(sorted([7, 3, 9]))  # [3, 7,
    ↪ 9]

def get_score(player):
    return player[1]

players = [("Mei", 40), ("Leo",
    ↪ 75)]
best = sorted(players,
    ↪ key=get_score,
              reverse=True)  # Leo
    ↪ 1st
```

Stack (last in, first out)

```
stack = []             # undo,
    ↪ plates
stack.append("circle")
stack.append("square")
last = stack.pop()     # "square"
```

Queue (first in, first out)

```
from collections import deque
queue = deque()        # turns, BFS
queue.append("Ada")
queue.append("Kai")
first = queue.popleft()   # "Ada"
```

Recursion

A function that calls itself on a **smaller** problem. It needs a **base case** to stop, or Python gives a `RecursionError`.

```
def tree(t, length):
    if length < 10:        # base
    ↪ case
        return
    t.forward(length)
    t.left(30)
    tree(t, length * 0.7) # smaller
    t.right(60)
    tree(t, length * 0.7)
    t.left(30)
    t.backward(length)     # go back
```

Grids (2D lists)

```
grid = [["."] * 5 for r in
    ↪ range(3)]
grid[1][2] = "#"          #
    ↪ [row][col]
for row in grid:
    print("".join(row))
```

Neighbors of `(r, c)`: up `(r-1, c)`, down `(r+1, c)`, left `(r, c-1)`, right `(r, c+1)`.

BFS (shortest path): put the start in a deque. Repeat: `popleft()` a square, add its unvisited neighbors to the back, remember where each came from.

CSV data

```
import csv
with open("games.csv", newline="",
          encoding="utf-8") as f:
    for row in csv.DictReader(f):
        text =
    ↪ row["rating"].strip()
        if text == "":
            continue      # skip
    ↪ blanks
        rating = int(text)  #
    ↪ text->int

# count with a dict:
counts[x] = counts.get(x, 0) + 1
```

git: save points (in Terminal)

`git init`	start a repo (once)
`git status`	what changed?
`git diff`	show changed lines
`git add game.py`	choose a file (`.` = all)
`git commit -m "Add X"`	make a save point
`git log --oneline`	list save points
`git restore game.py`	undo changes not yet committed
`pwd`	where am I?
`cd folder`	go into a folder

Good message: verb + what changed, e.g. `"Fix ghost walking through walls"`. Never commit secrets or real names.

pytest

```
from my_game import Grid  # no .py

def test_walls_block_moves():
    grid = Grid("###\n#P#\n###")
    assert grid.is_open(1, 1)
    assert not grid.is_open(0, 1)

def test_save(tmp_path):  # temp
    ↪ folder
    path = tmp_path / "scores.txt"
```

Run tests: Thonny **Tools** → **Open system shell…**, go to your folder with `cd`, then type `python3 -m pytest -q`.

Before you commit: review checklist

- ☐ It runs, and the tests pass.
- ☐ Clear names; one job per function.
- ☐ Comments explain why.
- ☐ No real names, passwords or photos.
- ☐ Kind feedback: two stars and a wish.